

Weekly Python Assignment 4 - Group 3 - S23

1(E). Define a function with three parameters that can add, subtract, multiply, or divide two numbers. The first parameter is a string variable that indicates the operation to be performed, such as "add" or "subtract". The second and third parameters are the two numbers to be operated upon. Return the result using a return statement, and then print the result. Test your function on the following operations:

2 + 3

4 - 2

3 * 2

3 / 2

3 ** 2

3 % 2

```
In [33]: def calculate(operation, num1, num2):
    if operation == "add":
        result = num1 + num2
    elif operation == "subtract":
        result = num1 - num2
    elif operation == "multiply":
        result = num1 * num2
    elif operation == "divide":
        result = num1 / num2
    elif operation == "power":
        result = num1 ** num2
    elif operation == "remainder":
        result = num1 % num2
    else:
        return "Error"
    return result
```

```
print(calculate("add", 2, 3))
print(calculate("subtract", 4, 2))
print(calculate("multiply", 3, 2))
print(calculate("divide", 3, 2))
print(calculate("power", 3, 2))
print(calculate("remainder", 3, 2))
```

5

2

6

1.5

9

1

2(M). Define a function called `list_analyzer` that accepts a single list-type parameter. Inside the function, print the number of elements, the maximum value, the minimum value, and the

average value in the inputted list. In each case, make sure you write text to indicate what you are printing, such as "The maximum value is: ". Test your function on these lists:

[1, 2, 3, 4, 5]

[2, 3, 1, 5, 4]

[10, 432, 32, -1, 2, 16, 111]

[-1, -3, -4, -5]

```
In [32]: def list_analyzer(lst):
    max_num = max(lst)
    min_num = min(lst)
    num_elements = len(lst)
    avg_num = sum(lst) / num_elements
    print(f"The number of elements in the list is {num_elements}")
    print(f"The maximum number is {max_num}")
    print(f"The minimum number is {min_num}")
    print(f"The average number is {avg_num}")

    lst1 = [1, 2, 3, 4, 5]
    lst2 = [2, 3, 1, 5, 4]
    lst3 = [10, 432, 32, -1, 2, 16, 111]
    lst4 = [-1, -3, -4, -5]

    list_analyzer(lst1)
    print("-----*5)
    list_analyzer(lst2)
    print("-----*5)
    list_analyzer(lst3)
    print("-----*5)
    list_analyzer(lst4)
```

The number of elements in the list is 5

The maximum number is 5

The minimum number is 1

The average number is 3.0

The number of elements in the list is 5

The maximum number is 5

The minimum number is 1

The average number is 3.0

The number of elements in the list is 7

The maximum number is 432

The minimum number is -1

The average number is 86.0

The number of elements in the list is 4

The maximum number is -1

The minimum number is -5

The average number is -3.25

3(E). Define a function called `vowel_finder` that accepts a single string-type parameter. This time, however, print the number of vowels first as indicated in the previous problem. After that, print "They are: " followed by the list of vowels in the string, separated by commas, all on one row. Call `vowel_finder` with the word "Shub-Niggurath" as input.

```
In [31]: def vowel_finder(word):
    vowel = ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
    result = []
    for i in word:
        if i in vowel:
            result.append(i)
    print(f"Number of vowels: {len(result)}")
    print("They are: ", result)

vowel_finder("Shub-Niggurath")
```

Number of vowels: 4
They are: ['u', 'i', 'u', 'a']

4(E). Open the file `input_data.txt` for reading. Open a second file `output_data.txt` for writing. Read the data from `input_data.txt` **line by line**, then write each line into `output_data.txt`. When you have finished, open `output_data.txt` for reading, read the entire file as a single string, and print the string.

```
In [30]: with open("input_data.txt", "r") as infile, open("output_data.txt", "w") as outfile:
    for line in infile:
        print(line)
        outfile.write(line)

with open("output_data.txt", "r") as infile2:
    for line in infile2:
        print(line)
```

1. this is just a short file.
2. We can use this to test different file operations.
3. Sometimes we only need a little bit of text.
4. That is why this file was created.
5. There are just a few lines in this file.
1. this is just a short file.
2. We can use this to test different file operations.
3. Sometimes we only need a little bit of text.
4. That is why this file was created.
5. There are just a few lines in this file.

5(M). Define a function that accepts a string-type parameter. The function should read in content of the file with the name designated by the inputted string parameter, then return the number of vowels in the text of that file. Call your function with the string "input_data.txt" as input.

```
In [29]: def read_content(file):
    vowel = ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
    result = 0
    with open(file, 'r') as infile:
        content = infile.read()
        for i in content:
            if i in vowel:
                result += 1

    return result

read_content('input_data.txt')
```

Out[29]: 60

6(H). Define a function that accepts a string-type parameter. In the function, begin by creating an empty list called `file_text`. Read the contents of the file **one line at a time**. Remember to strip off the carriage return at the end of each line. Use the `split()` command to create a list from the words in the line. Append this list to `file_text`. Do this for every line in the file. You now have a list (`file_text`) that contains all the lines from the file, each as a separate list. Print the text from `file_text`, making sure that you put a space between each word, so that your printing looks identical to the original file.

```
In [28]: def read_print(file):
    file_text = []
    with open(file, 'r') as infile:
        for line in infile:
            line = line.rstrip()
            words = line.split(" ")
            file_text.append(words)

    for content in file_text:
        print(" ".join(content))

read_print("input_data.txt")
```

1. this is just a short file.
2. We can use this to test different file operations.
3. Sometimes we only need a little bit of text.
4. That is why this file was created.
5. There are just a few lines in this file.

7(M). Create a CSV (.csv) file by opening a file called `numbers_out.csv` for writing. Create two lists with the following float values (not strings):

```
list1 = [23.0, 25.0, 27.0]
list2 = [37.1, 38.2, 38.5]
```

Write each list to `numbers_out.csv`, one line for each list. Remember that you can only write strings to a text file, so you will need to convert the list values for each list into a single string to write it as a line. When you have finished writing to the file, it should have two lines in it. Now

open `numbers_out.csv`, read the file **one line at a time**, and print each line to confirm that it was correctly saved.

```
In [27]: list1 = [23.0,25.0,27.0]
list2 = [37.1, 38.2, 38.5]
with open("numbers_out.csv", "w") as outfile:
    line1 = ','.join(map(str, list1))
    line2 = ','.join(map(str, list2))

    outfile.write(line1 + "\n")
    outfile.write(line2 + "\n")

with open("numbers_out.csv", "r") as infile:
    for line in infile:
        print(line)
```

23.0,25.0,27.0

37.1,38.2,38.5

8(H). Create an empty list called `my_data`. Read `numbers.csv` **one line at a time**. As you read each line of the file, split the line into a list, and then convert each string in the list to a float. Now append each list to `my_data`. When you have finished, `my_data` will have lists of floats, each corresponding to a line in the original file. Print `my_data` **one line at a time**, one line for each list, separating the float values with spaces (**not commas**).

```
In [26]: my_data = []
with open("numbers.csv", "r") as infile:
    for line in infile:
        line.rstrip()
        words = line.split(",")
        words = list(map(float, words))
        my_data.append(words)

for content in my_data:
    print(content)
```

[25.1, 25.2, 25.1]

[28.2, 28.3, 28.4]

[30.1, 30.0, 30.2]

9(M). Read in data from `numbers.csv`, then calculate and print the following:

- The overall maximum, minimum and average values in the file
- The index numbers for the lines with the maximum and minimum values, along with the values
- The index numbers for the lines having the highest and lowest average values, along with the values

```
In [17]: my_data = []
with open("numbers.csv", "r") as infile:
    for line in infile:
```

```

        line.rstrip()
        words = line.split(",")
        words = list(map(float, words))
        my_data.append(words)

overall_max = -1000000000000
overall_min = 1000000000000
lst_avg = []
lst_max = []
lst_min = []

total = 0 # for the overall average
elements = 0 # for the overall average

for content in my_data:
    total += sum(content)
    elements += len(content)
    each_avg = sum(content) / len(content)
    lst_avg.append(each_avg)
    lst_max.append(max(content))
    lst_min.append(min(content))

overall_avg = total / elements
print("The overall avg = %.2f" %overall_avg)
print("The overall max = %.2f" %max(lst_max))
print("The overall min = %.2f" %min(lst_min))
print("The highest value is in line %d with value %.2f" %(lst_max.index(max(lst_max))
print("The lowest value is in line %d with value %.2f" %(lst_min.index(min(lst_min)) +
print("The highest average value is in line %d with value %.2f" %(lst_avg.index(max(ls
print("The lowest average value is in line %d with value %.2f" %(lst_avg.index(min(lst

```

```

The overall avg = 27.84
The overall max = 30.20
The overall min = 25.10
The highest value is in line 3 with value 30.20
The lowest value is in line 1 with value 25.10
The highest average value is in line 3 with value 30.10
The lowest average value is in line 1 with value 25.13

```

10(M). Define a function that accepts two string-type parameters. The function will read in data from a file with the name provided by the first parameter, count how many instances of the string provided by the second parameter occurs in that content, and then print out the final count. Call your function using "tale.txt" and "best" as the first and second inputs, respectively.

Reminder: Do not include embedded occurrences where it is part of another word, such as "bestow".

```

In [25]: def read_count(file, word):
        total = 0
        with open(file, 'r') as infile:
            for line in infile:
                words = line.split()
                for i in words:
                    if i == word:
                        total += 1

```

```
    return total

print(read_count('tale.txt', 'best'))

40
```

```
In [24]: # Ignore the case of the alphabet
def read_count(file, word):
    total = 0
    with open(file, 'r') as infile:
        for line in infile:
            words = line.split()
            for i in words:
                check = i.lower()
                if check == word:
                    total += 1
    return total

print(read_count('tale.txt', 'best'))

40
```